

Software Requirements Specification & Project Planning Document

Coinly

Self-Hosted Personal Finance Tracker with AI Categorization, Insights, and Voice Q&A

Quantic School of Business and Technology
Master of Science in Software Engineering
MSSE Capstone Project

Version 1.0
Solo Project

Table of Contents

1. Introduction
 - 1.1 Purpose
 - 1.2 Project Scope
 - 1.3 Definitions, Acronyms, and Abbreviations
 - 1.4 References
2. Overall Description
 - 2.1 Product Perspective
 - 2.2 Product Functions
 - 2.3 User Classes and Characteristics
 - 2.4 Operating Environment
 - 2.5 Design and Implementation Constraints
 - 2.6 Assumptions and Dependencies
3. System Features (Functional Requirements)
 - 3.1 Data Foundation
 - 3.2 AI Categorization Engine
 - 3.3 Dashboard & Visualizations
 - 3.4 AI Insights & Anomaly Detection
 - 3.5 Budgets & Goal Tracking
 - 3.6 Q&A Chat with Voice Input
 - 3.7 Configuration & Preferences
4. Non-Functional Requirements
 - 4.1 Performance
 - 4.2 Reliability & Availability
 - 4.3 Security & Privacy
 - 4.4 Maintainability
 - 4.5 Usability & Accessibility
 - 4.6 Compliance
5. System Architecture
 - 5.1 High-Level Architecture
 - 5.2 Technology Stack
 - 5.3 Data Model
 - 5.4 Architectural Patterns
 - 5.5 Deployment Options
6. External Interface Requirements
 - 6.1 User Interfaces
 - 6.2 External APIs
7. Project Planning Structure

- 7.1 Agile Methodology
- 7.2 Roles (Solo Adaptation)
- 7.3 Product Backlog (User Stories)
- 7.4 Sprint Plan
- 7.5 Timeline & Milestones
- 8. Risk Register
- 9. Testing Strategy
- 10. Deployment & CI/CD Plan
- 11. Success Criteria & Rubric Alignment
- 12. Appendix

1. Introduction

1.1 Purpose

This document defines the Software Requirements Specification (SRS) and project plan for Coinly, a self-hosted personal finance tracker that ingests bank statement CSVs and manual transaction entries, uses AI to categorize transactions, surfaces spending insights, and answers natural-language questions about personal finances — including via voice input. The system is being developed as the capstone project for the Master of Science in Software Engineering (MSSE) program at Quantic School of Business and Technology.

This document serves as the authoritative specification for the project's functional behavior, technical architecture, sprint-based development plan, and success criteria. It is intended for the developer (acting as solo Product Owner, Scrum Master, and engineer), the Quantic faculty grader, and any future contributors reviewing the project repository.

1.2 Project Scope

Coinly is a privacy-first personal finance application designed for a single user to track, categorize, and understand their own spending and savings without sharing data with third-party financial aggregators. The application ingests transactions from two sources: CSV exports from common Egyptian banks (CIB, Banque Misr, NBE), and manual entry through the web UI. An AI categorization engine assigns each transaction to a category from a customizable taxonomy, learning from user corrections over time.

The dashboard presents spending patterns through interactive charts, trend lines, and category breakdowns. An AI insights engine generates plain-language weekly and monthly summaries highlighting notable changes, anomalies, and progress against user-defined budgets. A conversational Q&A interface — accessible via text or voice (browser Web Speech API) — allows the user to ask natural-language questions about their finances, with the system generating SQL queries against the local transaction database and returning structured answers.

Coinly is explicitly designed as a descriptive and analytical tool, not a financial advisory product. It does not provide investment recommendations, recommend specific purchases, or claim predictive certainty about future financial outcomes. All outputs are framed as observations and probabilistic forecasts with explicit uncertainty.

1.2.1 In Scope

- CSV import from Egyptian bank statement formats (CIB, Banque Misr, NBE) with extensible parser architecture
- Manual transaction entry with quick-add UX and recent-entry templates
- User-configurable base currency (EGP, USD, EUR, or any ISO 4217 code) with daily exchange-rate conversion
- AI-powered transaction categorization with learning from user corrections
- Interactive dashboard with spending breakdowns, trends, and recent-activity views
- AI-generated weekly and monthly insights and summaries
- Anomaly detection for unusual or large transactions
- Budget setting and progress tracking by category

- Q&A chat with natural-language queries, LLM-to-SQL execution, and voice input via Web Speech API
- Local-first architecture: SQLite database, no external accounts required for core functionality
- Optional cloud deployment with passcode protection and Turso (libSQL) for the public demo instance
- Comprehensive automated test suite and CI/CD pipeline

1.2.2 Out of Scope

- Direct bank API integration (Plaid, Tink, etc. — not available for Egyptian banks)
- Investment portfolio tracking, stock prices, or trading
- Bill payment, money transfer, or any transactional banking
- Receipt OCR or photo-to-transaction extraction
- Multi-user accounts, sharing, or social features
- Mobile native applications (responsive web only)
- Tax preparation, reporting, or filing
- Personalized investment or financial advice of any kind

1.3 Definitions, Acronyms, and Abbreviations

Term	Definition
SRS	Software Requirements Specification
MSSE	Master of Science in Software Engineering
CI/CD	Continuous Integration / Continuous Delivery
LLM	Large Language Model
RAG	Retrieval-Augmented Generation
LLM-to-SQL	Architectural pattern where an LLM generates SQL queries from natural-language questions, which are then executed against a database
Web Speech API	Browser-native JavaScript API for speech recognition and synthesis, supported in Chrome, Edge, and Safari
SQLite	Embedded single-file relational database
libSQL / Turso	Cloud-hosted SQLite-compatible database with persistent storage and replication
ORM	Object-Relational Mapper; in this project, Prisma
ISO 4217	International standard for currency codes (EGP, USD, EUR, etc.)

Anomaly	A transaction or pattern that deviates significantly from the user's historical norm
Categorization taxonomy	The hierarchical list of categories transactions are assigned to (e.g., Food → Groceries, Food → Dining Out)

1.4 References

- Quantic MSSE Capstone Handbook (Version dated 2/2/23)
- IEEE Std 830-1998: Recommended Practice for Software Requirements Specifications
- The Scrum Guide (Schwaber & Sutherland, 2020)
- Google Gemini API documentation (ai.google.dev/docs)
- Web Speech API specification (developer.mozilla.org/Web/API/Web_Speech_API)
- Turso / libSQL documentation (docs.turso.tech)
- Prisma ORM documentation (prisma.io/docs)

2. Overall Description

2.1 Product Perspective

Coinly is a self-contained, self-hostable web application. It is not part of a larger product suite and depends only on optional external services (exchange-rate API for currency conversion, Google Gemini API for AI features). The application is designed to operate fully offline once configured, with AI features gracefully degrading to non-AI alternatives when the LLM API is unavailable.

The application consists of three logical tiers: a Next.js frontend serving the user interface, an Express/Node.js backend exposing a REST API, and a SQLite (local) or libSQL (deployed) persistence layer accessed through the Prisma ORM. All three tiers can run on a single machine; the deployed configuration uses Turso for the database and Render or Fly.io for the application.

2.2 Product Functions

At a high level, Coinly performs the following functions:

1. Imports transactions from bank statement CSV files (multiple Egyptian bank formats supported)
2. Accepts manual transaction entry through a fast keyboard-driven UI
3. Converts transactions to the user's chosen base currency using daily exchange rates
4. Categorizes transactions into a customizable taxonomy using an LLM, with corrections that feed back into future predictions
5. Visualizes spending patterns through an interactive dashboard with multiple chart types
6. Generates plain-language weekly and monthly summaries highlighting trends, anomalies, and budget progress
7. Flags unusual transactions for user review
8. Allows the user to set and track budgets by category
9. Answers natural-language questions about transactions via LLM-to-SQL
10. Accepts voice input through the browser's Web Speech API for Q&A and quick-add

2.3 User Classes and Characteristics

User Class	Characteristics	Key Needs
Primary User (Self)	The developer using their own system to track their own finances. Technical user, comfortable with CSV exports.	Speed of entry, accurate categorization, useful insights, privacy
Reviewer / Grader	Quantic faculty evaluating the project. Time-constrained.	Working deployed instance, clear demo path, professional UI, code quality

Future Self-Hoster	Hypothetical user adopting the open-source project for personal use.	Clear setup docs, sane defaults, extensibility for other bank formats
--------------------	--	---

2.4 Operating Environment

- Client: Modern desktop browsers (Chrome, Edge, Safari) — Firefox supported with degraded voice features
- Server (local): Node.js 20+ on Windows, macOS, or Linux
- Server (deployed): Linux containers on Render or Fly.io
- Database (local): SQLite single file
- Database (deployed): Turso / libSQL (SQLite-compatible cloud)
- External services: Google Gemini API, exchangerate.host (free) or open.er-api.com for currency rates

2.5 Design and Implementation Constraints

- Solo developer: scope must be achievable by one person in approximately 14 weeks of part-time work
- Self-hosted-first: features must function with only a local installation; cloud is optional
- No persistent ephemeral storage: deployed instance uses Turso, not local-disk SQLite, because Render free-tier filesystems are wiped on restart
- Voice input limited to Web Speech API browsers (Chrome, Edge, Safari) — Firefox shows text-only mode
- LLM API costs: hard monthly spending cap required to prevent runaway expenses
- All code hosted in a GitHub repository shared with the quantic-grader account

2.6 Assumptions and Dependencies

- The user is comfortable exporting CSV statements from their online banking portal monthly
- Google Gemini API remains available with current pricing tier through the project lifetime
- Browser Web Speech API remains available in Chrome and Edge
- Egyptian bank CSV formats remain stable; updates require parser maintenance
- Free-tier exchange-rate APIs maintain current public access

3. System Features (Functional Requirements)

3.1 Data Foundation

3.1.1 Description

The system shall maintain a local database of all transactions, accounts, and supporting metadata, populated through both automated CSV import and manual entry.

3.1.2 Functional Requirements

- **FR-1.1:** The system shall parse CSV bank statements from the supported formats (CIB, Banque Misr, NBE) and import all rows as transactions with date, description, amount, currency, and source-account fields.
- **FR-1.2:** The system shall detect and skip duplicate transactions on re-import using a hash of date + amount + description + account.
- **FR-1.3:** The system shall provide a parser plugin interface so additional bank formats can be added without modifying core import logic.
- **FR-1.4:** The system shall accept manual transaction entry via a form supporting date, amount, currency, description, payee, account, and category.
- **FR-1.5:** The system shall support recurring transaction templates (e.g., rent, salary) that can be applied with one click.
- **FR-1.6:** The system shall maintain a daily exchange-rate table for the user's tracked currencies, refreshed daily from a free public API.
- **FR-1.7:** The system shall allow the user to edit or delete any transaction, with all changes logged for audit.
- **FR-1.8:** The system shall support multiple user-defined accounts (e.g., CIB Checking, NBE Savings, Cash, Credit Card) and track per-account balances.

3.2 AI Categorization Engine

3.2.1 Description

The system shall automatically assign each transaction to a category using an LLM-based classifier, with the ability to learn from user corrections.

3.2.2 Functional Requirements

- **FR-2.1:** The system shall provide a default category taxonomy covering common spending categories (Food, Transport, Bills, Entertainment, Health, Income, Transfers, etc.) with at least two levels of hierarchy.
- **FR-2.2:** The system shall allow the user to add, rename, merge, and delete categories.
- **FR-2.3:** The system shall call the Gemini API to classify newly imported transactions, with a structured prompt that includes the available taxonomy and a few-shot examples from the user's history.

- **FR-2.4:** The system shall batch transactions to minimize LLM API calls (target: ≤ 1 call per 20 transactions on average).
- **FR-2.5:** The system shall record the LLM's confidence in its assignment when available; transactions below a confidence threshold shall be flagged for manual review.
- **FR-2.6:** When the user corrects a category, the system shall add the correction to a rules table ("merchant X is always category Y") and apply the rule to future imports without invoking the LLM.
- **FR-2.7:** The system shall expose a categorization accuracy report comparing AI assignments against user corrections over time.
- **FR-2.8:** The system shall fall back to a rules-only categorization mode when the LLM API is unavailable, marking uncategorized transactions as "Uncategorized."

3.3 Dashboard & Visualizations

3.3.1 Description

The system shall provide an interactive dashboard as the primary user interface.

3.3.2 Functional Requirements

- **FR-3.1:** The dashboard shall display a summary card showing total income, total expenses, and net change for the selected period.
- **FR-3.2:** The dashboard shall display a category pie chart showing spending distribution for the selected period.
- **FR-3.3:** The dashboard shall display a monthly trend chart showing income and expenses over time.
- **FR-3.4:** The dashboard shall display a recent-transactions list with inline category editing.
- **FR-3.5:** The dashboard shall allow the user to drill down from any chart segment into the underlying transactions.
- **FR-3.6:** The dashboard shall support date-range filtering (This Month, Last Month, Last 3 Months, Year to Date, Custom Range).
- **FR-3.7:** All amounts shall display in the user's configured base currency, with original-currency tooltips on hover.
- **FR-3.8:** The dashboard shall be responsive on screens 360px wide and above.

3.4 AI Insights & Anomaly Detection

3.4.1 Description

The system shall surface notable patterns, anomalies, and AI-generated narrative insights to help the user understand their finances at a glance.

3.4.2 Functional Requirements

- **FR-4.1:** The system shall generate a weekly summary at the end of each calendar week, written in plain language, covering total spending, top categories, notable changes vs. prior week, and budget status.
- **FR-4.2:** The system shall generate a monthly summary at the end of each calendar month, with deeper trend analysis.
- **FR-4.3:** The system shall detect anomalous transactions using a z-score or IQR test on the per-category historical distribution and flag them in the UI.
- **FR-4.4:** Each AI-generated summary shall be tagged as such and dated, with the ability for the user to mark it as helpful or unhelpful (used for prompt iteration).
- **FR-4.5:** The system shall cache generated summaries to avoid duplicate LLM calls.
- **FR-4.6:** The system shall enforce a monthly LLM cost cap (configurable, default \$5/month). When the cap is reached, summaries fall back to a deterministic template-generated version.

3.5 Budgets & Goal Tracking

3.5.1 Description

The system shall support per-category budgets and track progress against them.

3.5.2 Functional Requirements

- **FR-5.1:** The system shall allow the user to set a monthly budget per category.
- **FR-5.2:** The system shall display a progress bar for each budget showing month-to-date spending vs. the budget.
- **FR-5.3:** The system shall flag budgets approaching (>80%) or exceeding their limit.
- **FR-5.4:** The system shall include budget status in the weekly and monthly AI summaries.

3.6 Q&A Chat with Voice Input

3.6.1 Description

The system shall provide a conversational interface for asking natural-language questions about the user's finances, with text or voice input.

3.6.2 Functional Requirements

- **FR-6.1:** The system shall provide a chat interface where the user can type or speak questions about their finances.
- **FR-6.2:** The system shall use the LLM-to-SQL pattern: the LLM receives the question, the database schema, and recent context, and generates a SQL query that is executed against a read-only database view.
- **FR-6.3:** All generated SQL shall be validated against an allowlist of safe SELECT statements; INSERT, UPDATE, DELETE, and DDL statements shall be rejected before execution.
- **FR-6.4:** The system shall present each answer in three formats: a plain-language sentence, a structured result table, and an expandable view of the generated SQL for transparency.

- **FR-6.5:** The system shall accept voice input via the Web Speech API in supported browsers, transcribing speech to the input field.
- **FR-6.6:** Voice quick-add: the user shall be able to dictate transactions ("add 250 pounds coffee from Costa") which the LLM parses into a structured transaction draft for user confirmation before saving.
- **FR-6.7:** The chat shall maintain short-term conversation context (last 5 turns) so follow-up questions like "and how about last month?" work naturally.
- **FR-6.8:** The system shall maintain an evaluation set of at least 30 natural-language questions with known correct answers, used in CI to measure LLM-to-SQL accuracy.

3.7 Configuration & Preferences

3.7.1 Description

The system shall provide a settings interface for user configuration.

3.7.2 Functional Requirements

- **FR-7.1:** The user shall be able to select their base currency from any ISO 4217 code, with a default of EGP.
- **FR-7.2:** The user shall be able to manage the category taxonomy (add, rename, merge, delete categories).
- **FR-7.3:** The user shall be able to manage accounts (add, rename, archive).
- **FR-7.4:** The user shall be able to set the monthly LLM cost cap.
- **FR-7.5:** The user shall be able to export all data as a single JSON file for backup.
- **FR-7.6:** The user shall be able to import a previously exported JSON file to restore data.
- **FR-7.7:** Switching base currency shall recalculate all displayed amounts; original-currency values shall be preserved unchanged in the database.

4. Non-Functional Requirements

4.1 Performance

- **NFR-1.1:** Dashboard pages shall render within 2 seconds for a database of up to 10,000 transactions.
- **NFR-1.2:** CSV import shall process at least 500 transactions per second on a typical laptop.
- **NFR-1.3:** AI categorization shall complete within 30 seconds for a batch of 100 transactions.
- **NFR-1.4:** Q&A responses shall return within 6 seconds end-to-end (LLM call + SQL execution + render) for typical queries.
- **NFR-1.5:** Voice recognition shall begin transcribing within 500ms of mic activation.

4.2 Reliability & Availability

- **NFR-2.1:** Local installations shall be available whenever the user's machine is running.
- **NFR-2.2:** The deployed demo instance shall maintain at least 95% availability.
- **NFR-2.3:** All AI features shall fail gracefully when the LLM API is unavailable, with the rest of the app continuing to function.
- **NFR-2.4:** The system shall maintain a transaction-change audit log enabling restoration of inadvertent edits.

4.3 Security & Privacy

- **NFR-3.1:** All API keys (Gemini, exchange-rate, etc.) shall be stored in environment variables, never committed to source control.
- **NFR-3.2:** The deployed demo instance shall be protected by a passcode (env var) checked on every API request.
- **NFR-3.3:** All deployed traffic shall be served over HTTPS.
- **NFR-3.4:** All LLM-generated SQL shall pass through a parser that rejects any non-SELECT statement before execution.
- **NFR-3.5:** Personal financial data shall never be sent to the LLM provider beyond what is strictly necessary; the system shall never include account numbers or full transaction descriptions in any prompt.
- **NFR-3.6:** Dependencies shall be scanned for known vulnerabilities on every CI run.

4.4 Maintainability

- **NFR-4.1:** Code shall achieve at least 70% line coverage from automated tests.
- **NFR-4.2:** All public functions and modules shall have TypeScript types and JSDoc comments.
- **NFR-4.3:** The repository shall include a comprehensive README with setup, configuration, and deployment instructions.
- **NFR-4.4:** Linting (ESLint), formatting (Prettier), and type checking (tsc) shall pass on every commit.

4.5 Usability & Accessibility

- **NFR-5.1:** The dashboard shall meet WCAG 2.1 Level AA contrast and keyboard navigation guidelines.
- **NFR-5.2:** Quick-add for transactions shall be fully keyboard-driven, completing in under 8 keystrokes for a typical entry.
- **NFR-5.3:** The system shall include a first-run setup wizard for configuring base currency, accounts, and initial categories.
- **NFR-5.4:** Numeric inputs shall use locale-appropriate thousands separators and decimal markers.

4.6 Compliance

- **NFR-6.1:** Every analytical view shall include a clearly visible disclaimer stating that the content is for personal informational use only and does not constitute financial advice.
- **NFR-6.2:** AI-generated text shall be labeled as such with a clear visual indicator.
- **NFR-6.3:** The repository's LICENSE file shall use a permissive open-source license (MIT or Apache 2.0).

5. System Architecture

5.1 High-Level Architecture

Coinly follows a layered architecture with clear separation of concerns:

Layer	Responsibility	Key Components
Presentation	User interface rendering and interaction	Next.js App Router pages, shadcn/ui components, Recharts
API	HTTP request handling, validation, passcode check	Express routes, Zod schemas, middleware
Application	Business logic orchestration	Import service, categorization service, insights service, Q&A service
Domain	Core business entities and rules	Transaction, Account, Category, Budget, Insight
Infrastructure	Persistence, external APIs, LLM client	Prisma repositories, Gemini client, exchange-rate client

5.2 Technology Stack

Category	Choice	Reasoning
Backend language	TypeScript 5 on Node.js 20	Type safety, large ecosystem, single language across stack
Web framework	Express 4	Mature, well-known, light footprint
Frontend framework	Next.js 14 (App Router)	Modern React with server components, fast dev experience
UI components	shadcn/ui + Radix + TailwindCSS	Polished defaults, accessible, no heavyweight dependencies
Charts	Recharts	React-native, composable, sufficient for dashboard needs
ORM	Prisma	Type-safe, excellent SQLite + libSQL support, great DX
Database (local)	SQLite (single file)	Zero-ops, perfect for single-user self-hosted
Database (deployed)	Turso / libSQL	SQLite-compatible cloud with persistence; matches local dev semantics

LLM provider	Google Gemini (gemini-2.0-flash, gemini-2.5-pro)	Generous free tier, fast, strong structured-output support
Voice input	Web Speech API (browser)	Free, no server cost, immediate UX with no infrastructure
Validation	Zod	Type-safe runtime validation aligned with TypeScript
Testing	Vitest + Playwright	Vitest for unit/integration, Playwright for E2E
CI/CD	GitHub Actions	Native to GitHub, free for public repos
Hosting	Render (web) + Turso (db)	Free tiers, simple deploys, persistent storage

5.3 Data Model

Core entities are summarized below. Foreign keys, indexes, and audit columns (created_at, updated_at) are omitted for brevity.

Entity	Key Fields	Purpose
accounts	id, name, type, currency, opening_balance, archived_at	User-defined accounts (bank, cash, credit, etc.)
transactions	id, account_id, date, amount_minor, currency, description, payee, category_id, ai_confidence, source, raw_csv_row	All transactions, with amount stored in minor units to avoid floating-point errors
categories	id, parent_id, name, color, icon, archived_at	Hierarchical categorization taxonomy
categorization_rules	id, match_type, pattern, category_id, created_from_correction	User-correction-derived rules applied before LLM
budgets	id, category_id, month, amount_minor, currency	Per-category monthly budget targets
exchange_rates	id, date, base, quote, rate	Daily rates for currency conversion
insights	id, period_start, period_end, type, content, model, generated_at	AI-generated weekly/monthly summaries
qa_history	id, asked_at, question, generated_sql, result_json, was_useful	Q&A audit trail and training signal
audit_log	id, entity, entity_id, action, before_json, after_json, timestamp	Change history for restorability

settings	key, value	Single-row key-value config (base currency, cost cap, etc.)
----------	------------	---

5.4 Architectural Patterns

- **Repository Pattern:** Data access is encapsulated in repository classes per aggregate, decoupling domain logic from Prisma specifics. This makes unit testing trivial and would allow swapping persistence in the future.
- **Strategy Pattern:** Categorization has multiple strategies (rule-based, LLM-based, hybrid) implementing a common interface and chained in priority order.
- **Service Layer:** Use cases (import_csv, categorize_batch, generate_weekly_insight, answer_question) are orchestrated by service classes that compose repositories and strategies. Controllers (Express routes) remain thin.
- **Adapter Pattern:** Bank CSV parsers implement a common BankStatementParser interface, allowing new bank formats to be added without modifying core import logic.
- **Pipeline Pattern:** CSV import is structured as a chain of steps (parse → deduplicate → normalize → categorize → persist), each independently testable.
- **Guarded LLM-to-SQL:** Q&A generates SQL through the LLM, but every query passes through a parser-based allowlist (only SELECT statements against known views) before execution — a pattern that protects against prompt injection.
- **Cost-Capped External Calls:** All LLM invocations route through a single client that tracks monthly spend against a configurable cap and short-circuits to fallback paths when exceeded.

5.5 Deployment Options

The application supports three deployment models, each appropriate for a different use case:

Option	Topology	Pros	Cons	Indicative Cost
Local Self-Host	SQLite file + Node on user's laptop	Maximum privacy, zero ops, fastest	No remote access, demo requires local recording	\$0
Cloud Demo (Recommended)	Render web service + Turso libSQL + passcode gate	Public demo URL for grader, persistent data	Requires passcode to access; small Turso usage	\$0 on free tiers
Fly.io with Volume	Fly app + persistent volume + SQLite	Cloud convenience with true SQLite	More setup, volume management	\$0–5/month
Production for Multi-User	Container + PostgreSQL + auth + WAF	Real production setup	Out of scope for this capstone	\$20–50/month

6. External Interface Requirements

6.1 User Interfaces

The web interface consists of the following primary screens:

- First-run setup wizard: base currency, primary account, taxonomy bootstrap
- Dashboard: summary cards, category pie, monthly trend, recent transactions
- Transactions: filterable, sortable, editable list of all transactions
- Import: CSV upload flow with preview, duplicate detection, and import-confirmation
- Quick-Add: keyboard-driven transaction entry
- Insights: chronological feed of AI-generated weekly and monthly summaries
- Budgets: per-category budget management and progress view
- Ask Coinly: chat interface with text + voice input
- Settings: base currency, categories, accounts, cost cap, data export/import
- About / Methodology: how categorization, insights, and Q&A work; LLM model used; cost tracking

6.2 External APIs

Provider	Purpose	Free Tier	Fallback
Google Gemini	Categorization, insights, Q&A SQL generation	Generous free tier with rate limits	Rules-only categorization; template summaries
exchangerate.host or open.er-api.com	Daily currency conversion rates	Unlimited public access	Cached most-recent rates
Web Speech API (browser)	Voice input	Browser-native, no cost	Text-only mode

7. Project Planning Structure

7.1 Agile Methodology

Development follows Scrum with two-week sprints. Each sprint produces a working increment, a demo recording for the simulated Product Owner, and a written retrospective. The project plans for five sprints plus a polish/submission phase, exceeding the Quantic Capstone Handbook minimum of three to allow for buffer and rework.

7.2 Roles (Solo Adaptation)

As a solo project, all Scrum roles are filled by the single developer, but kept conceptually distinct:

- **Product Owner:** Maintains the product backlog, prioritizes user stories, defines acceptance criteria. Documents decisions in writing to preserve perspective.
- **Scrum Master:** Runs sprint ceremonies on a written schedule, tracks blockers in the task board, enforces the Definition of Done.
- **Developer:** Implements user stories, writes tests, maintains the codebase.
- **Code Owner:** Self-reviews PRs with a checklist before merging; PRs remain open for at least 30 minutes to encourage genuine re-reading.

7.3 Product Backlog (User Stories)

User stories follow the format: As a [user type], I want [capability], so that [benefit]. Acceptance criteria use Given-When-Then. Estimates are in modified Fibonacci story points (1, 2, 3, 5, 8, 13).

7.3.1 Epic A — Data Foundation

ID	User Story	Priority	Est.
US-A1	As a developer, I want a configured Prisma schema so that data has a stable home.	High	3
US-A2	As a user, I want to import a CIB CSV statement so that I don't enter transactions manually.	High	8
US-A3	As a user, I want to import a Banque Misr CSV so that I can use my second bank.	High	5
US-A4	As a user, I want to import an NBE CSV so that I can use my third bank.	Medium	5
US-A5	As a user, I want duplicate transactions detected automatically so re-imports are safe.	High	3
US-A6	As a user, I want to add transactions manually so that cash spending is also tracked.	High	5

US-A7	As a user, I want to manage multiple accounts so I can track different sources.	High	5
US-A8	As a user, I want daily exchange rates so foreign transactions show in my base currency.	High	3

7.3.2 Epic B — AI Categorization

ID	User Story	Priority	Est.
US-B1	As a user, I want a default category taxonomy so I can start tracking immediately.	High	3
US-B2	As a user, I want to add, rename, and merge categories so the taxonomy fits my life.	High	5
US-B3	As a user, I want imported transactions auto-categorized by AI so I don't tag everything.	High	8
US-B4	As a user, I want to correct a category and have similar future transactions follow the correction.	High	5
US-B5	As a developer, I want LLM calls batched so categorization is cost-efficient.	High	3
US-B6	As a user, I want a rules-only fallback when the LLM is unavailable.	Medium	3
US-B7	As a user, I want a categorization accuracy report so I can see how the AI is performing.	Medium	5

7.3.3 Epic C — Dashboard & Visualizations

ID	User Story	Priority	Est.
US-C1	As a user, I want a summary card so I know my income, expenses, and net change at a glance.	High	3
US-C2	As a user, I want a category pie chart so I can see what I'm spending on.	High	5
US-C3	As a user, I want a monthly trend chart so I can see direction over time.	High	5
US-C4	As a user, I want to drill down from charts to underlying transactions.	Medium	5
US-C5	As a user, I want to filter by date range so I can focus on any period.	High	3

US-C6	As a user, I want the dashboard to work on my phone so I can check while out.	Medium	5
-------	---	--------	---

7.3.4 Epic D — Insights & Anomalies

ID	User Story	Priority	Est.
US-D1	As a user, I want a weekly AI summary so I understand my recent spending in plain language.	High	5
US-D2	As a user, I want a monthly AI summary with deeper trend analysis.	High	5
US-D3	As a user, I want unusual transactions flagged so I notice them quickly.	Medium	5
US-D4	As a developer, I want LLM costs capped and tracked so I don't get a surprise bill.	High	3

7.3.5 Epic E — Budgets

ID	User Story	Priority	Est.
US-E1	As a user, I want to set a monthly budget per category.	High	3
US-E2	As a user, I want progress bars showing budget consumption.	High	3
US-E3	As a user, I want warnings when a budget is approaching or exceeded.	Medium	2

7.3.6 Epic F — Q&A Chat & Voice

ID	User Story	Priority	Est.
US-F1	As a user, I want to ask natural-language questions and get correct answers.	High	8
US-F2	As a developer, I want generated SQL validated against an allowlist before execution.	High	5
US-F3	As a user, I want to see the SQL behind the answer for transparency.	Medium	2
US-F4	As a user, I want to ask questions by voice so I can use it hands-free.	High	5
US-F5	As a user, I want to dictate transactions ("add 250 EGP coffee") for quick capture.	Medium	5

US-F6	As a developer, I want a Q&A evaluation set so I can measure accuracy over time.	High	5
-------	--	------	---

7.3.7 Epic G — Settings, Quality & Deployment

ID	User Story	Priority	Est.
US-G1	As a user, I want a first-run setup wizard so I'm productive immediately.	Medium	5
US-G2	As a user, I want to change my base currency at any time.	Medium	3
US-G3	As a user, I want to export and re-import all my data as JSON for backup.	Medium	3
US-G4	As a developer, I want CI running tests, lint, and type checks on every push.	High	5
US-G5	As a developer, I want automated deployment to Render on merge to main.	High	3
US-G6	As a developer, I want at least 70% test coverage on core logic.	High	8
US-G7	As a developer, I want a comprehensive README and design document.	High	5

7.4 Sprint Plan

Solo velocity is targeted at approximately 25–30 points per two-week sprint, with deliberate buffer for the demanding day-job constraint.

Sprint 0 (Week 1) — Setup & Discovery

- Initialize GitHub repository, share with quantic-grader
- Set up Trello board with the full backlog
- Scaffold Next.js + Express + Prisma + SQLite
- Configure CI pipeline skeleton (lint, type-check, build, test)
- Provision Turso database, Gemini API key, secrets in env
- Write Product Owner persona document and Definition of Done
- Document architecture diagrams in the design doc starter

Sprint 1 (Weeks 2-3) — Data Foundation

Sprint Goal: Import CSVs from CIB and Banque Misr, add manual transactions, view them in a basic list. Healthy CI on every push.

- US-A1, US-A2, US-A3, US-A5, US-A6, US-A7

- US-G4 (initial CI pipeline)
- Sprint demo: import two CSVs, add a manual transaction, view the merged list with no duplicates

Sprint 2 (Weeks 4-5) — Categorization + Dashboard MVP

Sprint Goal: AI categorization working end-to-end. First version of the dashboard live.

- US-A4, US-A8
- US-B1, US-B2, US-B3, US-B4, US-B5
- US-C1, US-C2, US-C5
- US-G5 (deployment to Render + Turso)
- Sprint demo: deployed Coinly with AI-categorized transactions and a working pie chart

Sprint 3 (Weeks 6-7) — Insights, Budgets, Anomalies, Trends

Sprint Goal: Make Coinly genuinely useful — insights, budgets, anomaly flags, monthly trends.

- US-B6, US-B7
- US-C3, US-C4
- US-D1, US-D2, US-D3, US-D4
- US-E1, US-E2, US-E3
- Sprint demo: first AI weekly summary, budget bars, anomaly callouts

Sprint 4 (Weeks 8-9) — Q&A Chat + Voice

Sprint Goal: Conversational Q&A with text and voice working reliably. Evaluation harness in place.

- US-F1, US-F2, US-F3, US-F4, US-F5, US-F6
- US-G6 (test coverage push)
- Sprint demo: text + voice Q&A live on the deployed instance, eval report showing measured accuracy

Sprint 5 (Weeks 10-11) — Polish, Settings, Mobile

Sprint Goal: First-run wizard, settings, data export, mobile responsive, accessibility.

- US-C6, US-G1, US-G2, US-G3
- Accessibility audit + fixes
- Performance pass: dashboard < 2s with 10k transactions
- Sprint demo: full responsive UI, polished settings, accessibility checks passing

Weeks 12-14 — Documentation, Demo & Submission

- US-G7: finalize README and DESIGN.md
- Write methodology page explaining AI categorization, insights, and Q&A
- Record final 15–20 minute demo with government ID shown
- Final bug fixes and polish from a fresh-eyes review

- Submit deliverable links via Quantic dashboard

7.5 Timeline & Milestones

Week	Milestone	Sprint
1	Project bootstrapped, backlog ready	Sprint 0
3	End of Sprint 1: importing CSVs, manual entry works	Sprint 1
5	End of Sprint 2: AI categorization + dashboard deployed	Sprint 2
7	End of Sprint 3: insights + budgets + anomalies live	Sprint 3
9	End of Sprint 4: Q&A chat + voice + evaluation	Sprint 4
11	End of Sprint 5: feature-complete, mobile-ready	Sprint 5
12	Check-in Week consultation with Quantic faculty	Polish
13	Design doc finalized, demo recorded	Polish
14	Submission to Quantic dashboard	Submission

8. Risk Register

Risks scored on Likelihood and Impact, each 1 (low) to 5 (high). Risk Score = Likelihood × Impact.

Risk	L	I	Score	Mitigation
Day-job demands compress development time	5	4	20	Strict sprint goals; defer non-MVP items to a stretch list; finish polish features last; plan ~30% buffer per sprint
Gemini API limits or pricing change	2	4	8	Abstract LLM client; rules-only categorization fallback; template-summary fallback; provider-swap path documented
LLM-to-SQL generates incorrect queries	4	3	12	Allowlist SELECT-only against known views; evaluation set with measured accuracy; SQL transparent to user in the UI
Egyptian bank CSV formats vary unpredictably	3	3	9	Plugin parser architecture; per-format unit tests with fixture CSVs; manual fallback always available
SQLite + Render free-tier ephemerality breaks demo	3	4	12	Use Turso libSQL for deployed instance; SQLite only for local
Voice features inconsistent across browsers	4	2	8	Document supported browsers; degrade gracefully to text-only in Firefox; clear UI affordance
Categorization accuracy disappoints	3	3	9	Frame as "assisted" not "automatic"; user corrections feed back; rules engine handles the easy cases
LLM costs spike unexpectedly	2	4	8	Hard monthly cap configured in code and Gemini console; daily cache; alert on threshold
Scope creep — adding investment tracking, receipts, etc.	4	4	16	Out-of-scope list is enforceable; new ideas go to a separate "v2" backlog, not the sprint
Plagiarism concern from reused snippets / AI assistance	1	5	5	Cite external code in CITATIONS.md; document AI assistance use in AI_USAGE.md; ensure all submitted code is personally written and understood

9. Testing Strategy

Testing is structured as a pyramid: many fast unit tests, fewer integration tests, and a small number of end-to-end tests covering critical user journeys. The strategy also includes a domain-specific evaluation harness for the LLM-to-SQL feature.

9.1 Unit Tests

- CSV parsers (pure functions; one fixture file per supported bank format)
- Categorization rules engine (matching logic, conflict resolution)
- Currency conversion and rounding (especially edge cases around minor-unit storage)
- Anomaly detection statistics
- Budget calculation logic
- SQL allowlist validator (positive and negative cases — must reject INSERT, UPDATE, DELETE, DDL, multi-statement, comments)
- Repository classes (using in-memory SQLite test database)
- Zod schemas (validation success and failure cases)

9.2 Integration Tests

- End-to-end CSV import: file upload → parse → deduplicate → categorize (mocked LLM) → persist
- API endpoints against a test database
- LLM client with the cost cap (mocked LLM, asserts fallback when cap exceeded)
- Q&A pipeline: question → SQL generation (mocked) → allowlist → execution → response shape

9.3 LLM-to-SQL Evaluation Harness (Project-Specific)

A dedicated test suite, run weekly in CI, evaluates LLM-to-SQL accuracy. The evaluation set contains at least 30 hand-written questions with known correct answers against a fixture database. Metrics tracked over time:

- Exact-match accuracy of returned numeric/categorical answers
- Tolerance-match for floating-point answers
- SQL syntactic validity rate
- SQL allowlist pass rate
- End-to-end success (question in → correct answer out, regardless of SQL form)

Results are committed to docs/evals/ as a JSON file with dated entries, providing an honest accuracy trail.

9.4 End-to-End Tests (Playwright)

- First-run setup wizard completes successfully
- Import a CSV, see transactions appear with categories

- Correct a category, see the rule applied to a similar new transaction
- Set a budget, exceed it, see the warning
- Ask a Q&A question, see a non-empty answer
- Dashboard loads correctly on a 360px viewport

9.5 Manual Test Cases

Maintained in a test plan covering: visual review, accessibility (keyboard nav, screen reader sanity), real-bank CSV smoke tests with actual data (in a separate non-CI environment), browser compatibility for voice features.

9.6 Tooling

Tool	Purpose
Vitest	Unit and integration tests
@vitest/coverage-v8	Coverage reporting
Playwright	End-to-end browser tests
ESLint + @typescript-eslint	Linting
Prettier	Code formatting
tsc --noEmit	Static type checking
npm audit + Snyk	Dependency vulnerability scanning
GitHub Actions	Orchestration of all of the above on every push

10. Deployment & CI/CD Plan

10.1 Branching Strategy

- Trunk-based development on main
- Short-lived feature branches named feature/US-XX-short-description
- Pull requests required for every change to main, even solo, to exercise CI
- Linear history enforced (squash and merge)

10.2 CI Pipeline (every push and PR)

11. Checkout code
12. Set up Node 20 with caching
13. Install dependencies (npm ci)
14. Run ESLint and Prettier --check
15. Run tsc --noEmit
16. Run npm audit (high severity blocks)
17. Run Vitest with coverage; fail if coverage drops below 70%
18. Run Prisma migrations against test SQLite
19. Build Next.js production bundle
20. Run Playwright E2E against the built app
21. Upload coverage and eval reports as artifacts

10.3 CD Pipeline (merge to main)

22. All CI checks must pass
23. Build container image and tag with commit SHA
24. Push to Render via deploy hook
25. Run smoke tests against the deployed URL
26. Roll back automatically if smoke tests fail

10.4 Scheduled Jobs

Job	Schedule	Purpose
refresh_exchange_rates	Daily 03:00 UTC	Pull latest currency rates
generate_weekly_insight	Sunday 22:00 UTC	Generate the weekly AI summary
generate_monthly_insight	1st of month 22:00 UTC	Generate the monthly AI summary

weekly_qa_eval	Sunday 04:00 UTC	Run the LLM-to-SQL evaluation set and log results
nightly_smoke_test	Daily 04:30 UTC	Hit health and key endpoints; alert on failure

11. Success Criteria & Rubric Alignment

The Quantic Capstone Handbook scores submissions on a 0–5 scale; a 5 requires addressing all requirements with outstanding execution and demonstrated initiative beyond the minimum. The table below maps deliverables to rubric criteria.

Rubric Criterion	How Coinly Satisfies It
GitHub repository with documented code	Public repo shared with quantic-grader. README, CONTRIBUTING, architecture diagram, JSDoc, TypeScript types throughout.
Deployed Web application	Render deployment of full stack with Turso backend, protected by passcode for the grader.
Up-to-date task board	Trello board with full epic/story breakdown, sprint columns, real-time updates throughout development.
Detailed design & testing document	Dedicated DESIGN.md and TESTING.md covering architectural patterns, technology rationale, deployment options & cost analysis, complete test strategy, and LLM-to-SQL evaluation methodology.
Appropriate engineering methodology	Five completed Scrum sprints with planning, review, retrospectives, and Definition of Done. Trunk-based branching with PR reviews.
CI/CD tooling	GitHub Actions running lint, type check, tests, build, E2E, and security scan on every commit; automated deploys to Render.
AI tooling demonstrated	Gemini for categorization, insights, and Q&A. LLM-to-SQL with measured accuracy via weekly evaluation harness. Voice input via Web Speech API. Cost-capped LLM client with graceful fallback.
Outstanding demonstration	15–20 minute walkthrough showing live deployed app: CSV import, AI categorization, dashboard, insights, voice Q&A. Voice in the demo is a high-impact moment.
Initiative — above and beyond	LLM-to-SQL pattern with allowlist guardrail (rare in student projects), weekly LLM-to-SQL evaluation harness (rarer still), voice input integration, multi-currency support, cost-capped AI usage, accessibility compliance, plugin parser architecture.

11.1 Definition of Done

A user story is Done when:

- Code implemented and merged to main via a reviewed PR

- Unit and integration tests written and passing in CI
- Lint, format, and type checks pass
- Acceptance criteria from the Trello card demonstrably met
- Relevant documentation updated
- If user-facing: deployed to the live Render instance
- If user-facing: manual UX smoke test performed

12. Appendix

12.1 Glossary of Key Patterns

- **LLM-to-SQL:** A pattern where natural-language questions are translated into SQL queries by an LLM that knows the database schema. The query is then executed against the database, and the result is returned (often re-formatted by the LLM). More reliable than RAG for aggregations because actual computation is performed.
- **Allowlist SQL Validation:** A defense-in-depth pattern where LLM-generated SQL must pass through a parser that confirms it is a SELECT statement against known tables/views before execution. Protects against prompt injection and accidental destructive operations.
- **Cost-Capped LLM Client:** A wrapper around an LLM SDK that tracks cumulative monthly spend, blocks further calls when a configured cap is reached, and routes blocked calls to fallback paths (rule-based, template, or cached responses).
- **Repository Pattern:** Encapsulation of data access logic behind a domain-friendly interface, decoupling business logic from the persistence layer.
- **Plugin Parser:** A common interface implemented by per-format bank-statement parsers, allowing new bank formats to be added without modifying core import logic.

12.2 Trello Board Structure

- Product Backlog — all unestimated and lower-priority stories
- Sprint Backlog — committed stories for the current sprint
- In Progress — currently active work (WIP limit: 2)
- In Review — code complete, awaiting self-review or CI checks
- Blocked — items waiting on external factors, with reason documented
- Done — completed in the current sprint
- Archive — completed stories from previous sprints

12.3 LLM Prompting Conventions

Coinly uses structured prompts with the following conventions:

- Every prompt is versioned and stored in prompts/ in the repo (not in code)
- Prompts use explicit role markers (system, user, assistant) and avoid relying on implicit context
- Categorization prompts include a short list of few-shot examples from the user's own history (anonymized to merchant + amount range)
- SQL-generation prompts include the database schema, a sample of the data, and explicit instructions to return only SQL (no commentary)
- Insight-generation prompts include aggregated numerical data, never raw transactions

12.4 AI Assistance Disclosure

This project will use AI tools (LLMs, code assistants) during development for tasks including: generating boilerplate code, drafting documentation, suggesting test cases, and reviewing pull requests. All such use will be documented in the repository's AI_USAGE.md file in accordance with academic integrity policies. The developer remains responsible for the correctness, security, and design of all code merged to main.

12.5 Document History

Version	Date	Author	Changes
1.0	Initial Draft	Solo Developer	Initial SRS and planning document for the MSSE Capstone